Jsp - Servlet

Chapitre 3: Récupération d'informations

Dr. Ahmad Khoureich Ka

UADB

L3 D2AW

1. Introduction
2. Les paramètres d'initialisation
3. Les informations contenues dans la requête
   - Les données de formulaires
   - Informations sur la machine cliente
   - Informations sur l'utilisateur
   - Les en-têtes HTTP de requête
4. Les informations sur le serveur
5. Autres informations
   - Attributs de requête
   - Attributs de contexte
   - Transfert de requête
6. Objets implicites

Introduction

- Pour bien répondre à la requête du client une application web doit pouvoir accéder à certaines informations :

1. Les paramètres d'initialisation.
2. Les informations contenues dans la requête :
   - données de formulaires,
   - spécificités du client
   - en-têtes HTTP de requête.
3. Les informations sur le serveur :
   - caractéristiques de la machine serveur,
   - caractéristiques du logiciel serveur.

Les paramètres d'initialisation

1. **Paramètre d'initialisation de servlet**
   - Défini dans `web.xml` ou à l'aide d'annotations dans le servlet.
   - N'est accessible que pour le servlet pour lequel il est défini.
   - Récupérable à l'aide d'un objet `ServletConfig` (`javax.servlet.ServletConfig`)

2. **Paramètre d'initialisation de contexte**
   - Défini dans `web.xml`.
   - accessible à tous les servlets et jsp de l'application.
   - Récupérable à l'aide d'un objet `ServletContext` (`javax.servlet.ServletContext`)

Paramètre d'initialisation de servlet

☞ Exemple définition dans **web.xml**

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app ...>
   ...
   <servlet>
      <servlet-name>HtmlServlet</servlet-name>
      <servlet-class>htmlservlet.HtmlServlet</servlet-class>
      <init-param>
         <param-name>nbLignes</param-name>
         <param-value>5</param-value>
      </init-param>
   </servlet>
   ...
</web-app>
```

Paramètre d'initialisation de servlet

- Exemple définition à l'aide d'annotations

```
@WebServlet(
        name = "HtmlServlet",
        urlPatterns = {"/HtmlServlet", "/HServlet"},

        initParams = {
           @WebInitParam( name = "nbLines", value = "5" ),
           @WebInitParam( name = "nbRows", value = "45" )
        }
)
public class HtmlServlet extends HttpServlet {
    ...
}
```

- Récupération du paramètre d'initialisation de servlet
  Méthode de la classe `javax.servlet.GenericServlet`

```
String nbRows = getServletConfig().getInitParameter("nbRows");
```

Les paramètres d'initialisation de contexte

- Exemple définition dans **web.xml**

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app ...>
    <context-param>
        <param-name>dbUrl</param-name>
        <param-value>jdbc:mysql://localhost:3306/scol</param-value>
    </context-param>
    ...
</web-app>
```

- Récupération du paramètre d'initialisation de context
  Méthode de la classe **javax.servlet.GenericServlet**

```
String dbUrl = getServletContext().getInitParameter("dbUrl");
```

Les paramètres d'initialisation

Remarque

- Les objets **`ServletConfig`** et **`ServletContext`** disposent de la méthode :

  `Enumeration<String> getInitParameterNames()`

  qui retourne les noms des paramètres d'initialisation (associés respectivement au servlet et au contexte).

Exemple

- Affichage de la liste des paramètres d'initialisation du contexte.

```
Enumeration<String> listParam =
    getServletContext().getInitParameterNames();
while(listParam.hasMoreElements()){
    out.println(listParam.nextElement());
}
```

Les informations contenues dans la requête

Ces informations regroupent :

- les données de formulaires,
- les informations sur la machine cliente,
- les informations sur l'utilisateur,
- les en-têtes HTTP de requête

Les données de formulaires

- Les formulaires permettent à l'utilisateur d'échanger des informations avec l'application.

- Les données de formulaires sont intégrées à la requête :
  - avec la méthode `get` où, elles sont ajoutées à la fin de l'url après le point d'interrogation.
  - avec la méthode `post` où elles sont séparées de l'url.

Les données de formulaires

Requête **GET** et requête **POST**

- requête **GET**

```
GET /nom_du_servlet?param1=valeur1&param2=valeur2&... HTTP/1.1
Host: localhost:8080
User-Agent: Mozilla/...
Accept: text/html,...
Referer: ...
  ...
```

- requête **POST**

```
POST /nom_du_servlet HTTP/1.1
Host: localhost:8080\r\n
User-Agent: Mozilla/...
Accept: text/html,...
  ...
param1=valeur1&param2=valeur2&...
```

Les données de formulaires

- Un paramètre de requête peut ne pas avoir de valeur ou avoir une ou plusieurs valeurs.

- Les paramètres de requête sont récupérés avec les mêmes méthodes :
  - `getParameter(String nom)`
  - `getParameterValues(String nom)`
  - `getParameterNames()`

  de l'interface `javax.servlet.http.HttpServletRequest`

getParameter(String nom)

- La méthode

```
public String getParameter(String name)
```

est utilisée pour les paramètres monovalués. Elle prend en argument le nom du paramètre dont elle doit retourner la valeur.

- La valeur retournée est :
  - une chaine de caractères (`String`) si le paramètre a une valeur,
  - une chaine de caractères vide si le paramètre n'a pas de valeur,
  - `null` si le paramètre n'existe pas dans la requête.

Remarque :
Il est conseillé de tester l'existence de la valeur retournée par `getParameter` avant de l'utiliser.

getParameter(String nom)

Exemple :

Dans un formulaire, un champ de saisi :

```
Nom: <input type='text' name='nom'/>
```

Récupération de la valeur saisie :

```
String nom = request.getParameter("nom");
```

getParameterValues(String nom)

- La méthode

```
public String[] getParameterValues(String name)
```

est utilisée pour les paramètres multi-valués. Elle prend en argument le nom du paramètre dont elle doit retourner les valeurs.

- Cette méthode retourne :
  - toutes les valeurs du paramètre dans un tableau de chaine de caractères (`String[]`),
  - un tableau ne contenant que des chaines de caractères vides si le paramètre n'a aucune valeur,
  - `null` si le paramètre n'existe pas dans la requête.

Remarque :
Il est conseillé de tester l'existence de la valeur retournée par `getParameterValues` avant de l'utiliser.

getParameterValues(String nom)

Exemple :

Dans un formulaire, trois champs de saisi :

```
Donnez 3 fruits<br/>
<input type='text' name='fruits' /> <br/>
<input type='text' name='fruits' /> <br/>
<input type='text' name='fruits' /> <br/>
```

Récupération des valeurs :

```
String[] fruits = request.getParameterValues("fruits");
```

getParameterNames()

- La méthode

```
public Enumeration<String> getParameterNames()
```

permet d'obtenir la liste des paramètres contenus dans la requête.

- Exemple : quelques champs d'un formulaire :

```
Prénom: <input type='text' name='prenom'/><br/>
Nom: <input type='text' name='nom'/><br/>
Civilité:
<input type='radio' name='civil' value='h'/>M.
<input type='radio' name='civil' value='mme'/>Mme
<input type='radio' name='civil' value='mlle'/>Mlle
```

Affichage de la liste des paramètres :

```
Enumeration<String> listParam = request.getParameterNames();
while(listParam.hasMoreElements()){
   out.println(listParam.nextElement());
}
```

Les informations sur le client

- Les informations sur le client concernent l'utilisateur et la machine ayant émis la requête. Ces informations sont obtenues à partir de méthodes de `HttpServletRequest`

- Les informations sur la machine cliente sont :
  - son adresse ip, récupérée avec la méthode

    ```
    public String getRemoteAddr()
    ```

  - son nom (nom de domaine complètement qualifié ou FQDN), récupéré avec la méthode

    ```
    public String getRemoteHost()
    ```

Remarque : Si la requête est envoyée par un serveur proxy, ces méthodes retournent respectivement l'adresse ip ou le nom de celui ci.

Les informations sur le client

- Les informations sur l'utilisateur sont :
  - le type d'authentification mis en œuvre pour l'identifier, récupéré avec la méthode

    ```
    public String getAuthType()
    ```

    Cette méthode retourne :
    - **BASIC_AUTH** authentification login/mot de passe avec la boite de dialogue du navigateur ;
    - **DIGEST_AUTH** authentification basique avec mot de passe transformé par hachage ;
    - **FORM_AUTH** authentification login/mot de passe avec un formulaire crée par le développeur ;
    - **CLIENT_CERT_AUTH** authentification HTTPS (par certificat client).
    - **null** si aucune authentification n'est utilisée.

Les informations sur le client

- username utilisé lors de l'authentification :

```
public String getRemoteUser()
```

  Cette méthode retourne `null` si l'utilisateur n'est pas authentifié.

- tester si le client possède le rôle spécifié :

```
public boolean isUserInRole(String role)
```

- username et ensembles des rôles du client authentifié :

```
public Principal getUserPrincipal()
```

  Cette méthode retourne `null` si l'utilisateur n'est pas authentifié.

Actions sur le client

- L'application web peut exécuter des actions sur l'utilisateur qui a émis la requête :
  - Demander à l'utilisateur de s'authentifier

```
boolean authenticate(HttpServletResponse response)
```

Cette méthode retourne `false` si l'utilisateur n'est pas enregistrer.

  - Authentifier l'utilisateur en se basant sur le username et le mot de passe qu'il a fourni :

```
void login(String username, String password)
```

Cette méthode lance une exception `ServletException` lorsque `username` ou `password` est invalide.

  - Effacer les informations concernant l'utilisateur authentifié.

```
void logout()
```

Les en-têtes HTTP de requête

Structure de la requête :

- requête GET

```
GET /nom_du_servlet?param1=valeur1&param2=valeur2&... HTTP/1.1
Host: localhost:8080
User-Agent: Mozilla/...
Accept: text/html,...
Referer: ...
  ...
```

- requête POST

```
POST /nom_du_servlet HTTP/1.1
Host: localhost:8080...
User-Agent: Mozilla/...
Accept: text/html,...
  ...
param1=valeur1&param2=valeur2&...
```

Les en-têtes HTTP de requête

- Les en-têtes HTTP de requête sont des informations jointes à la requête par le navigateur.
- Ils sont parfois utiles pour le traitement de la requête.
- Les en-têtes de requête sont définis par le protocole HTTP.

Les en-têtes HTTP de requête

- Les en-têtes HTTP les plus utilisés :
  - **`Accept`**,indique les formats de données que peut gérer le navigateur (**`"text/html"`**, **`"text/plain"`**, **`"image/gif"`**, **`"image/jpeg"`**, ...) ;
  - **`Accept-Language`**, indique par ordre de préférence les langues acceptées par client ;
  - **`Accept-Charset`**, indique le jeu de caractères acceptée par le client ;
  - **`Accept-Encoding`**, indique le type d'encodage de contenu supporté par le navigateur ;
  - **`Host`**, indique le nom du serveur ou son adresse ip, suivi du port comme spécifiés dans l'url ;

Les en-têtes HTTP de requête

- Les en-têtes HTTP les plus utilisés (suite) :
  - `User-Agent`, indique le navigateur utilisé par le client ;
  - `Referer`, indique l'url de la page ayant permis d'avoir la page courante ;
  - `Cookie`, spécifie une donnée (un cookie) à envoyer au serveur.
  - `Authorization`, contient le username et le mot de passe (encodés en base64) fournis pas l'utilisateur.

Méthodes pour la lecture des en-têtes de requête

Les en-têtes HTTP sont lus à l'aide de méthodes de `javax.servlet.http.HttpServletRequest` :

- `public String getHeader(String name)`, retourne la valeur :
  - de l'en-tête nommé,
  - `null` si l'en-tête ne figure pas dans la requête.
- `public long getDateHeader(String name)`, retourne la valeur :
  - de l'en-tête nommé,
  - `-1` si l'en-tête ne figure pas dans la requête.

  Exemple l'en-tête `If-Modified-Since` indique la date de dernière modification de la réponse.

Méthodes pour la lecture des en-têtes de requête

- `public int getIntHeader(String name)`, retourne la valeur
  - de l'en-tête nommé,
  - `-1` si l'en-tête ne figure pas dans la requête.

  Exemple : `content-length` indique en nombre d'octets la taille du fichier (ou des fichiers) présent(s) dans la requête.

- `public Enumeration<String> getHeaders(String name)` retourne l'ensemble des valeurs de l'en-tête multi-valué spécifié.

- `public Enumeration<String> getHeaderNames()`, retourne l'ensemble des noms d'en-têtes contenus dans la requête.

Les en-têtes de requête

Exemple :

Lire tous les en-têtes HTTP contenus dans la requête :

```
Enumeration<String> enTetes = request.getHeaderNames();
while (enTetes.hasMoreElements()) {
    String nomEntete = enTetes.nextElement();
    String valeurEntete = request.getHeader(nomEntete);
    out.println(nomEntete + ": " + valeurEntete + "<br/>");
}
```

Les informations sur serveur

Informations sur le serveur :

- nom de la machine serveur :

```
// Méthode de HttpServletRequest
public String getServerName()
```

- port d'écoute :

```
// Méthode de HttpServletRequest
public int getServerPort()
```

- nom et la version du logiciel serveur :

```
// Méthode de ServletContext
public String getServerInfo()
```

Autres informations

- IL y a des informations qui peuvent être attachée à la requête ou au contexte de l'application :
  - les attributs de requête,
  - les attributs de contexte.

- C'est un mécanisme qui permet de partager des informations avec des servlets ou jsp.

Attributs de requête

- Attacher un objet à la requête :

```
// Méthode de HttpServletRequest
void setAttribute(String nom, Object objet)
```

- Récupérer un objet (attribut) attaché à la requête :

```
// Méthode de HttpServletRequest
Object getAttribute(String nom)
```

Remarque

Cette méthode retourne `null` lorsque l'attribut n'existe pas dans la requête.

Attributs de requête

- Supprimer un attribut précédemment attaché à la requête :

```
// Méthode de HttpServletRequest
void removeAttribute(String nom)
```

- Obtenir la liste des attributs attachés à la requête :

```
// Méthode de HttpServletRequest
Enumeration<String> getAttributeNames()
```

Attributs de contexte

- Attacher un objet au contexte de l'application :

```
// Méthode de ServletContext
void setAttribute(String nom, Object objet)
```

- Récupérer un objet (attribut) précédemment attaché au contexte de l'application :

```
// Méthode de ServletContext
Object getAttribute(String nom)
```

Remarque

Cette méthode retourne `null` lorsque l'attribut n'existe pas dans le contexte.

Attributs de contexte

- Obtenir la liste des attributs attachés au contexte de l'application

```
// Méthode de ServletContext
Enumeration<String> getAttributeNames()
```

- Supprimer un attribut précédemment attaché au contexte de l'application :

```
// Méthode de ServletContext
void removeAttribute(String nom)
```

Transfert de requête

- Une requête peut être transférée d'une ressource à une autre.
- Les attributs attachés à la requête restent disponibles après le transfert.
- Le transfert d'une requête peut se faire à l'aide :
  d'un objet `javax.servlet.RequestDispatcher`.
  ou
  de l'action standard `jsp:forward`

Transfert de requête

Exemple 1

Transférer une requête à partir d'un servlet

```
// Dans un servlet
RequestDispatcher rd = request.getRequestDispatcher("url_cible");
rd.forward( request, response );
```

Exemple 2

Transférer une requête à partir d'une page jsp

```
// Dans une page jsp
<jsp:forward page="url_cible" />
```

Transfert de requête

Remarques :

- Les attributs attachés à la requête ne sont disponibles que durant le traitement cette dernière.
- L'url donnée à la méthode `getRequestDispatcher` est relative au contexte de l'application.

Transfert de requête

- Il est possible d'ajouter des paramètres à la requête avant le transfert.

Exemple 1

Avec `RequestDispatcher`

```
// Dans un servlet
RequestDispatcher rd =
    request.getRequestDispatcher("url_cible?param1=valeur1&...");
rd.forward( request, response );
```

Exemple 2

Avec l'action standard `jsp:forward`

```
// Dans une page jsp
<jsp:forward page="url_cible" >
    <jsp:param name="param1" value="valeur1"/>
    ...
</jsp:forward>
```

Quelques objets implicites jsp

- request : représente la requête (`HttpServletRequest`).
- response : représente la réponse (`HttpServletResponse`).
- out : représente l'objet permettant d'écrire sur la réponse (`PrintWriter`).
- application : représente le contexte de l'application (`ServletContext`).
- config : représente l'objet `ServletConfig` du servlet obtenu à partir de la page jsp.

Les objets implicites

- pageContext : représente la page jsp. Des attributs peuvent être attachés puis être récupérés de `pageContext`.

Exemple

- `pageContext.setAttribute(nom, object)`
- `pageContext.getAttribute(nom)`

- exception : représente l'exception lancée par une page jsp et reçue par une page d'erreur.

Exemple

```
<%@ page isErrorPage="true" %>
L'erreur suivante est survenue: <%= exception %>
```